Mobiclip Middleware for TWL

Quick start guide

Introduction

The Mobiclip Middleware for TWL is divided into different components:

- The Mobiclip Tools for NINTENDO DS that contains:
  - The AviToMobiclipDs conversion tool.
  - The Mobiclip Video for Windows(VFW) codec.
- Video resources.
- Five code samples.
- Mobiclip library and prototype files.

Each of these components will be described below.

History

* Version 1.2.2 (September 3rd, 2010)

+ Added 'Mobiclip_Tools_for_DS.exe' self-installer. It installs the Mobiclip VFW codec and the AviToMobiclipDS.exe program.
+ Modifications done in Mobiclip VFW codec:
  * StarForce protection system dropped, replaced by Mobiclip license system.
  * installation using .inf file dropped, now installed by 'Mobiclip_Tools_for_DS.exe' self-installer.
  * GUI has changed.
  * New features added, some have been removed; please read 'Mobiclip Video for Windows (VFW) codec description' chapter for more information.
+ Updated QuickStart and MobiclipFAQ documents.
+ Following documents removed:
  * AboutLogo.pdf
  * Starforce protection - Administration Guide.pdf
  * Starforce protection - Automatic Activation.pdf
  * Starforce protection - Manual Activation.pdf
  * VX To Mobiclip Shifting Guide.pdf
+ Modifications done in html reference documentation in MO_JumpNextIframe.html: described precise behaviour of the MO_JumpNextIframe function.

* Version 1.2.1 (October 22nd, 2009)

+ Removed bug in AviToMobiclipDS program: converted movies were showing audio artifacts at playback when Adobe Flash program was used to generate uncompressed Avi file(used later as source for video compression).

+ Updated QuickStart document: added batch job usage using VirtualDub V1.9.6 in 'Multipass Encoding Howto' chapter.

+Added Makefile in libMobiclip/build/demos/Mobiclip directory, used to compile all demos at once.

+Modifications done in html reference documentation in singleScreen.html, singleScreenStereo.html, dualScreen.html and effectiveUsage.html files: main.srl renamed to the real demo name in 'Compile Method' section (singleScreen.srl for example for the singleScreen demo).

+Added 'CPU requirements for Mobiclip VFW codec' section in readme.txt file.

* Version 1.2.0 (April 23rd, 2009)

+ Developpement environment changed from NitroSDK to TwlSDK.
+ Modifications done to singleScreen, singleScreenStereo, dualScreen and multitrackAudio demos: sound was not playing for all TWL HYBRID and LIMITED builds.
+ Modified build system of sample codes:
    * CodeWarrior projects have been deleted.
    * Build process is done now using makefiles in the same manner as the sample codes in the TwlSDK.
+ New libMobiclip libraries are added for TWL HYBRID and LIMITED targets:
    * libMobiclip.TWL.HYB.a
    * libMobiclip.TWL.LTD.a
+ Updated html reference documentation about the new ways to compile the sample codes for TWL target.
+ Updated QuickStart document about the support of the new TWL targets and their related libraries.

* Version 1.1.1 (March 9th, 2009)

+ Modifications done in effectiveUsage demo: removed a bug that could make the playback loop forever if the decode is too slow.
+ Modifications done to singleScreen, singleScreenStereo, dualScreen and multitrackAudio demos:
    - modifications to make the NINTENDO DS cover management match the specification of NINTENDO DS hardware:
        * Added GX_DispOff/GXS_DispOff function calls before calling PM_SetLCDPower with 'PM_LCD_POWER_OFF' as parameter.
        * Added GX_DispOn/GXS_DispOn function calls after having called PM_SetLCDPower with 'PM_LCD_POWER_ON' as parameter.
    - modifications to allow playback of movies containing less than 'MO_NB_BUFFERED_FRAME_MIN' frames.
+ Added function to AviToMobiclipDS application:
    - Now always display an error when input audio is not PCM 16 bits.
    - Is able to read BWF WAV format, sound must be PCM 16 bits mono or stereo.
    - Removed a crash with VirtualDub generated WAV files given as input.
    - Program version is now V1.2.0.2.

* Version 1.1.0 (November 21st, 2008)
    + Modified sample demos:
        - singleScreen, singleScreenStereo and dualScreen demos modified because no processing was done when DS closed during the end of movie playback.
    + Added function to AviToMobiclipDS application
        - Added IMA ADPCM and PCM support.
        - Use multiple [audio] options to specify different tracks. Doing multi-track audio video playback is now possible with more than 2 audio tracks.
        There is no number of audio track limitation.
        Each audio track consumes about 5.2KB of main memory.
            Limitations:
            - All audio tracks must have the same frequency.
            - All audio tracks must have the same number of channels.
            - If more than one [audio] option is used, [debugsound] option cannot be used.
    + Updated QuickStart
        - Added documentation about the new audio formats.
    + Updated Reference manual(Html)
        - Added restriction usage of MO_SkipFrameSoundOnePacket.
    + Added Sample code.
        - Added multitrackAudio sample code. It shows how to dynamically change the current played audio track.

* Version 1.0.3 (September 22nd 2008)
    + Upgraded Starforce protection so the Video For Windows codec can be installed and used on Windows Vista.
    + Updated Starforce documentation:
        - Added section about serial number associated limitations
        - Updated screenshots to match the new starforce version
        - Added section about renewing codec
    + Updated Reference manual(Html)
        - Added forgotten links in the a-z function listing.

* Version 1.0.2 (May, 7th 2008)
    + Added function to AviToMobiclipDS application
        - By selecting [audio] option external wav file can be used for sound source.
          Application version is now V1.1.0.0.
    + Sample code has been modified.
        - when the following conditions where met at the same time in sample code of
          singleScreen, singleScreenStereo, and dualScreen, arithmeticbug occurred.
          It has been fixed:
                * There was a sound buffer underflow.
                * The sound playback was paused and restarted.
    + Added document to the package
        - 'VX To Mobiclip Shifting Guide.pdf'.
                * How to port a libVX application to Mobiclip has been explained.
        - 'MobiclipFAQ.pdf'.
                * Support information associated or can be used to mobiclip from
                  questions related to VX middleware has been explained.
                  A Frequently Asked Questions document.
    + Added function to Mobiclip Video Codec For Windows.
        - By disabling keyframe detection you can avoid inserting key frame automatically.

Version 1.0.1 (March, 14th 2008)
    + Sample code modifications:
        - define 4KB of stack to free up DTCM space.
        - use OS_GetDTCMArenaHi and OS_GetDTCMArenaLo to compute available
          DTCM space.
        - added OS_CheckStack call after MO_OpenMovie call to check that stack is not
          overwritten by Mo_OpenMovie function.
    + .Html documentation updated:
        - added information about the need of specific stack allocation in lcf files to free up
          DTCM space for Mobiclip usage).
        - all references to ‘SDRAM memory’ replaced by ‘main memory’.

Version 1.0 (March, 3rd 2008)
    + Initial release

Mobiclip SDK for TWL general features

Video playback features:

- During single-screen display, up to 30 frames/second for full screen (256x192) playback
- During two-screen display, up to 15 frames/second for full screen (256x192) playback on both screens
- Bit rate ranges from 100 to 700 Kbps/screen

Audio playback features :

- Supports both mono and stereo audio
- Frequency ranges from 8 to 48 KHz
- Data rates:
    - For mono tracks, the rate ranges from 0.6 to 16 bits/sample
    - For stereo tracks, the rate ranges from 1.2 to 32 bits/sample

The Mobiclip SDK includes support for multiple audio codecs. Depending on your requirements concerning the compression ratio, the quality and the decoding speed, you can choose the most appropriate using the following specifications:

Codec	Sound Quality	Data Rate (for each audio track)	Frequency	Bit Depth	Decode Speed
Standard	Average	0.6～1.25bits/sample	8～48KHz	16bit	Average
FastAudio(FA)	Better than Ave.	1.25 bits/sample（Fixed）	8～48KHz	16bit	1/2 of Average
ADPCM (IMA-ADPCM)	Better	4 bits/sample（Fixed）	8～48KHz	16bit	Faster
PCM	Best	16 bits/sample（Fixed）	8～48KHz	16bit	Fastest

Additionally, a movie can include multiple audio tracks using the “Multi-track Function”. For details, please refer the “Sound Preparation” within “Video and Audio Processing”.

Mobiclip Tools for NINTENDO DS

Introduction

These are composed of two video utilities:

- The Mobiclip Video for Windows(VFW) codec that will be used for encoding video sources into Mobiclip format. The target file will be an AVI file containing Mobiclip video frames and optionally audio tracks.
- AviToMobiclipDs that is a program used to convert an AVI file containing Mobiclip video frames into the Mobiclip file format. This target format will be read by the libMobiclip library on the DS platform, its file extension is .mods.

Installation

Step 1 - Self-installer usage

- Go to the libMobiclip/tools directory.
- Double-click on file Mobiclip_Tools_for_DS.exe.
- Just follow the installation instructions of the self-installer

NOTE: The self-installer performs a silent uninstall of existing tools before beginning the installation process.

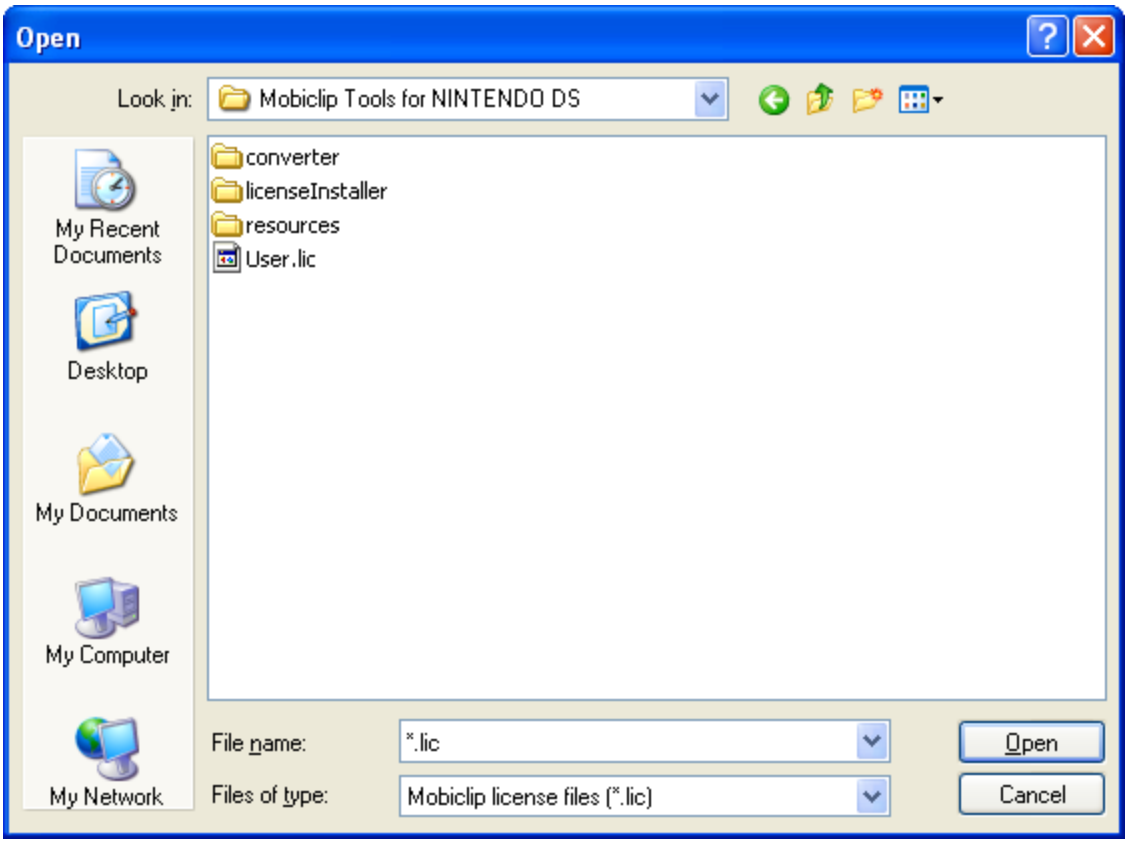
NOTE: The self-installer automatically adds/removes at install/uninstall the directory containing AviToMobiclipDS.exe tool in the user PATH environment variable.

Step 2 - Installation of a valid Mobiclip license

A Mobiclip license consists of a file named *User.lic*; this file contains various informations about the rights to use the Mobiclip Tools for NINTENDO DS and its duration validity.
The Mobiclip Tools for NINTENDO DS does not include a valid Mobiclip license. You need to obtain it separately. Please contact Nintendo technical support to know how to obtain it.

Below you will find the steps needed to install a Mobiclip license.

- From the Windows Start menu, go to All Programs->Mobiclip Tools for NINTENDO DS
- Click on Install a new Mobiclip license.



- Browse your computer to select your stored User.lic license, and click on Open.

Your Mobiclip license is now installed, you can later check the license duration validity by using the About dialog box of the Mobiclip VFW codec, either from an encoder tools or from the Windows Start menu, by going to *All Programs->Mobiclip Tools for NINTENDO DS* and clicking on *About Mobiclip VFW codec for DS*.



Video and audio processing

Video editing tool

The video edit tool is necessary to use Mobiclip Video Codec for Windows and to compress the video.

We recommend using the freeware VirtualDub as a video tool to process your videos. You can download it at http://www.virtualdub.org/.

You can also use "off the shelf" products from other software vendors, but the following sections explain the encoding process using VirtualDub as an example.

Compatibility issues

Some applications do not conform to the Video For Windows 1.1 standard (VFW 1.1) because they do not provide the codec with the frames per second information or the total number of frames information.

The codec detects such an incompatible application, and displays a popup that tells the user what information the codec lacks.

Known incompatible applications:

- Adobe After Effects CS3: it doesn't give the Frames Per Second information nor the Number of Frames information. This encoding mode is the only known working mode:
  - Single Pass – Constant Quality
- Adobe Premiere Pro CS3, this product doesn't give the Number of Frames information. These encoding modes are the only known working modes:
  - Single Pass – Constant Quality
  - Multipass – VBR - First Pass
  - Multipass – VBR - Nth Pass

Video Preparation and Processing

Video preparation and processing are very important. They will contribute to 50% of the final video quality, the remaining 50% coming from the video codec itself.

Rule #1: Use the best video source you can get

Rule #2: Resolution of the video sources should be big, at least 640x480
Many processes (like contrast/gamma) will benefit from the size of the video and create errors (after the final downsize), because they are applied to more pixels beforehand.

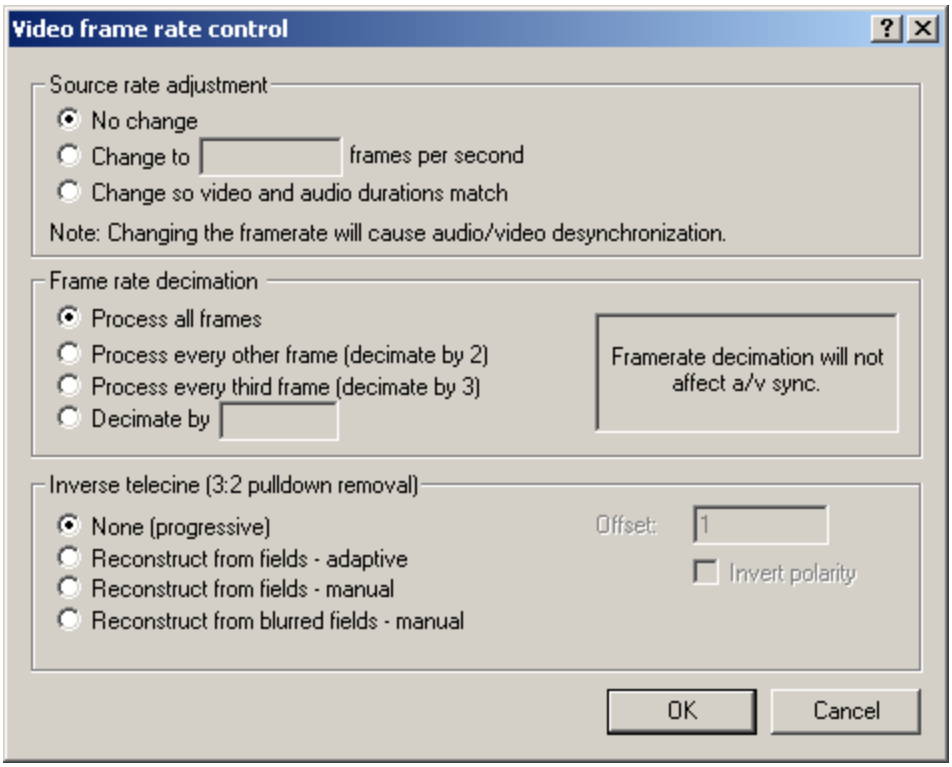
Rule #3: Delay the final downsizing to DS resolution as late as possible in your processing pipeline

Rule #4: Source video should be uncompressed or not compressed much

Rule #5: Be careful of the source video frame rate

Typical videos captured from TV are 30 fps. However, the original frame rate (cinema) is often 24 fps. In that case you should convert the movie back to 24 fps with an “inverse telecine” method to ensure better quality. This kind of method is available in VirtualDub in the *Video/Frame rate* menu.

Rule #6: When lowering frame rate, only use "decimation" methods
See the *Video/Frame rate* menu in VirtualDub.



Rule #7: Do your best to minimize any changes in the aspect ratio of the source and final video (in general 10% difference is ok)
Your source videos will probably have an aspect ratio of 4/3 (TV), 16/9 (cinema) or 2.35 (cinema also). DS aspect ratio is 4/3 (256/192). To convert from source aspect ratio to DS aspect ratio, you will have to crop your video (remove a part of the image) or add black bars.

Rule #8: Downsize using a "Precise bicubic" (or better) method
You can choose the target resolution of the videos with the following restrictions:
- Final resolution must not exceed 256x192 for DS.
- Width and height of the target resolution must be multiples of 16.

Again, do not forget to preserve the original aspect ratio.

Rule #9: When storing intermediate movies, use a lossless video codec
We recommend using Huffyuv for storing intermediate files.
See http://neuron2.net/www.math.berkeley.edu/benrg/huffyuv.html.

Mobiclip Video for Windows (VFW) codec description

Access the VFW codec configuration menu

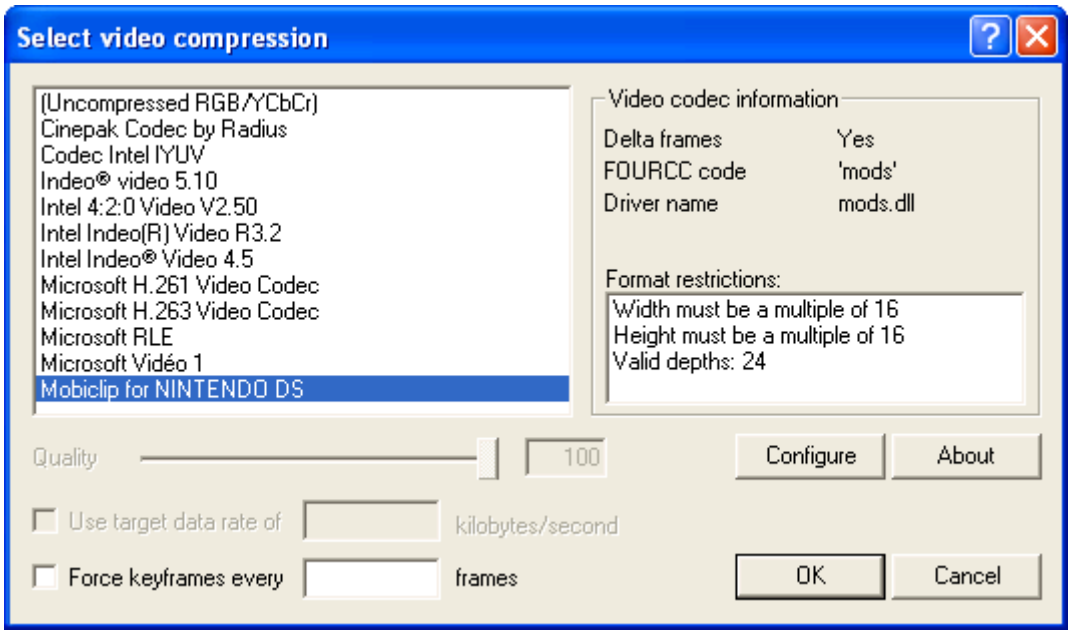
You have two ways to have access to the configuration menu of the Mobiclip Video for Windows codec:

- From the Windows Start menu, go to All Programs->Mobiclip Tools for NINTENDO DS and click on Mobiclip VFW codec for DS Configuration.
- From a video editing tool.

Below is described a way to configure the Mobiclip Video for Windows codec from VirtualDub program.

Selecting the Mobiclip VFW codec for DS

VirtualDub, open the *Video → Compression* menu and choose M**obiclip for NINTENDO DS**:



Opening the Mobiclip VFW codec for NINTENDO DS configuration dialog

Click on the *Configure* button.

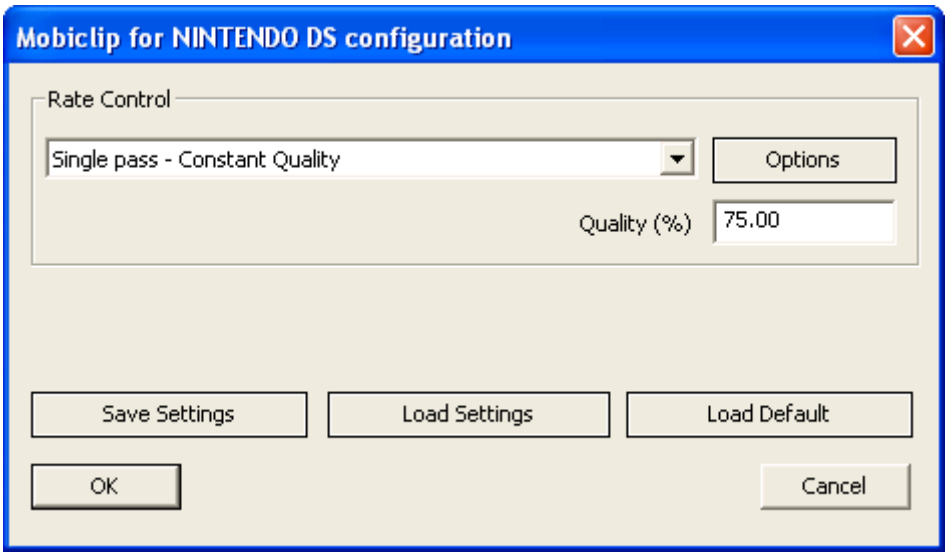
Tuning the parameters of the Mobiclip VFW codec for NINTENDO DS

Set the parameter values.

You will find a complete description of the configuration dialog.

Mobiclip Video Codec for Windows options description

The following dialog box appears when the ‘Configure’ button is pressed:

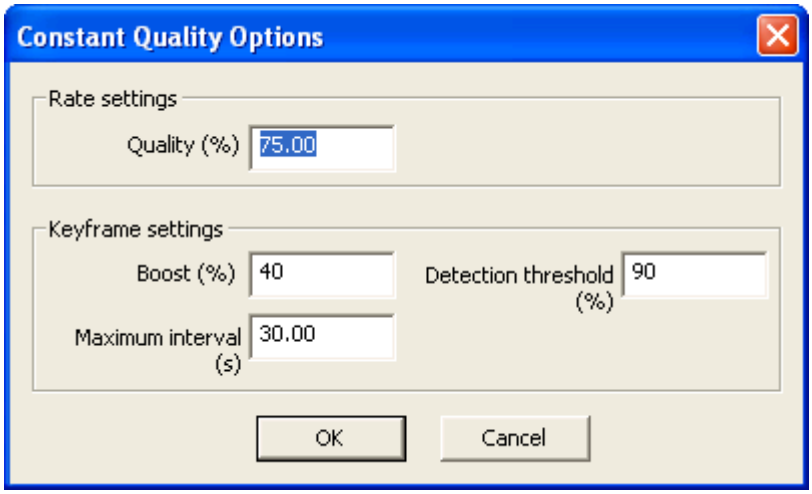


Encoding Type

In the Rate Control section you can choose what type of encoding algorithm you want to use. There are five different choices:

Single Pass – Constant Quality

This is the default method. The video is encoded based upon the quality parameter. Valid values are in the range 0 to 100, and the default value is 75. This method is only available in a Single Pass encoding type. When Options button is pressed the following dialog box appears:



- **Quality:**
  The quality parameter described before.

- **Boost:**
  !!! WARNING: don't modify this parameter except if you are an expert user !!!
  This parameter is used to boost the quality of the encoded keyframes of the video. This often results in a quality increase in the video sequences after keyframes, and its default value is 40.

- **Detection threshold:**
  This is a tuning parameter for scene change detection. The default value is 90, which means that if more than 90% of the image has changed it will be flagged as a keyframe.
  If you have static parts in your video images like permanent black band, you should lower this value. For example if there is only 70% of your images which is changing (30% are permanently black) you should put 70% * 90% = 63% as the parameter.

- **Maximum Interval:**
  This is the maximum interval where the video encoder is forced to put keyframes in your video. Valid values (in seconds) are in the range 0 to 3600 and the default value is 30 seconds, which means that the video encoder will be forced to put a keyframe in your video every 30 seconds interval, even if no scene change has been detected.

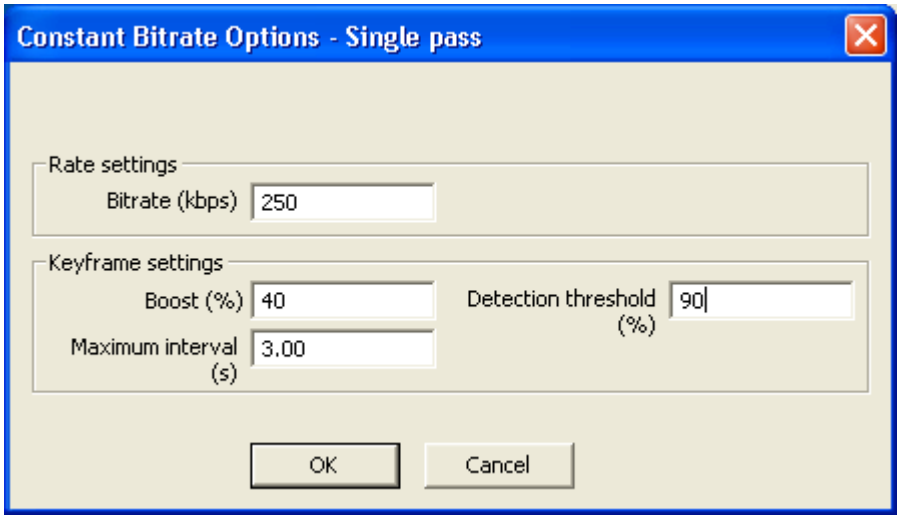
Single Pass – CBR

The video encoder uses single pass constant bitrate encoding type. The Bitrate parameter is given in kilobits per second, valid values are in the range 1 to 20000 and the default value is 250.

This encoding mode lets you target a specific file size by setting the bitrate obtained from the following formula:

Average bitrate (kbps) / 8 * duration (seconds) = video target size (KBytes)

When Options button is pressed the following dialog box appears:



- **Bitrate:**
  The bitrate parameter described before.

- **Boost, Detection threshold, Maximum Interval:**
  They have the same meaning as in the *Single Pass – Constant Quality* encoding scheme. The default value for *Maximum interval* is 3 seconds.

Multipass - VBR - First Pass

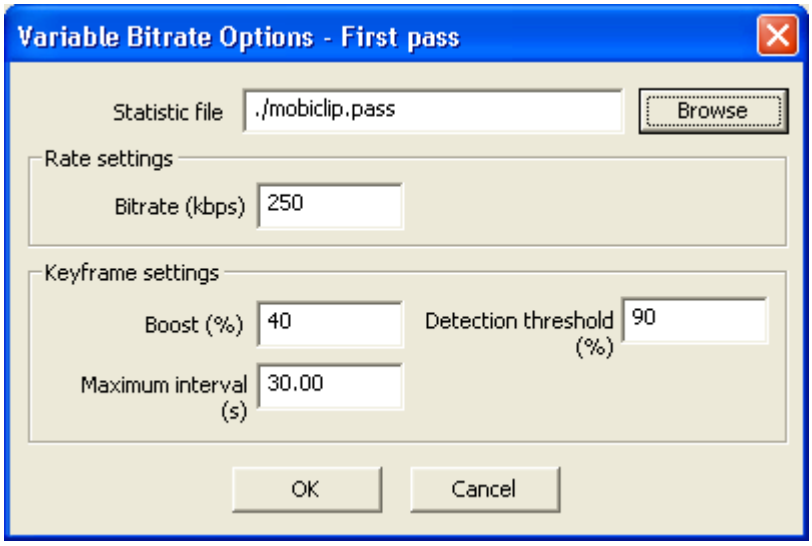
The first pass of a variable bitrate multipass video encoding type is used to gather statistics about the video to optimize the next passes
(Multipass - VBR - Nth Pass).

The multipass VBR encoding scheme is useful when you want the best video quality for a given target size. Given the duration of the movie and the desired target size, you can calculate the corresponding average bitrate value that is required.

Average bitrate (kbps) / 8 * duration (seconds) = video target size (KBytes)

The *Bitrate* parameter is given in kilobits per second, valid values are in the range 1 to 20000 and the default value is 250.

When *Options* button is pressed the following dialog box appears:



- **Stat File:**
  This lets you configure the location and the name of the file which will hold the statistics used for multi-pass encoding. The default file is './mobiclip.pass'.

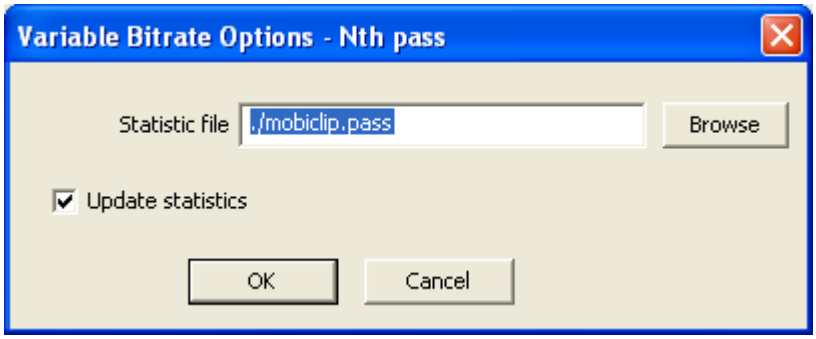
- **Bitrate:**
  The bitrate parameter described above.

- **Boost, Detection threshold, Maximum Interval:**
  They have the same meaning as in the *Single Pass – CBR* encoding scheme.

Multipass - VBR - Nth Pass

The successive passes of a variable bitrate multipass video encoding type are used to optimize the video quality, and are performed after the first pass.

This process can be executed any number of times to better target the desired average bitrate or filesize. It should be executed at least once to have a usable video target, and if you call it twice you will be very close to the targeted bitrate. When *Options* button is pressed the following dialog box appears:



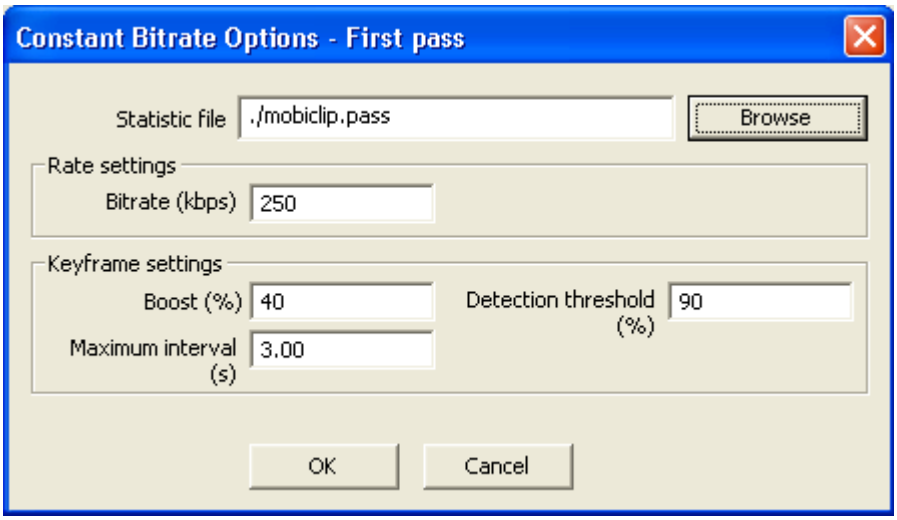
- **Stat File:**
  This is the same parameter described previously.

- **Update Statistics:**
  Check this box if you want the encoding statistics to be updated during this pass, and it is checked by default. This option is useful if you want to encode a movie using the *Multipass – VBR - Nth Pass* encoding type more than one time.

Multipass - CBR - First Pass:

The first pass of a constant bitrate multipass video encoding type is used to gather statistics about the video to optimize the next passes (Multipass - CBR - Nth Pass.) This encoding scheme will give you better video quality than the CBR single pass scheme.

The *Bitrate* parameter is given in kilobits per second, valid values are in the range 1 to 20000 and the default value is 250. When *Options* button is pressed the following dialog box appears:

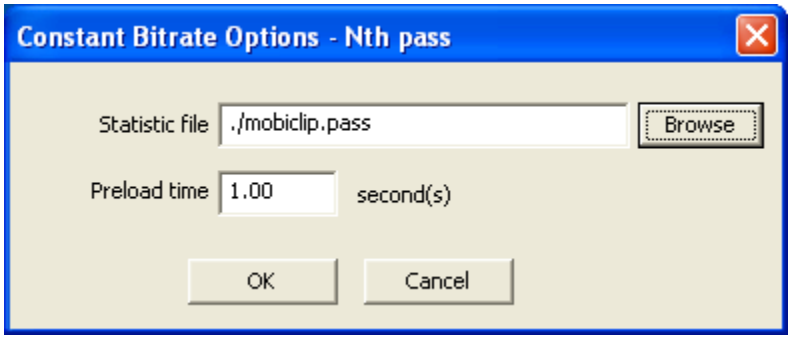


- **Stat File, Bitrate, Boost, Detection threshold, Maximum Interval:**
  They have the same meaning as described before.

Multipass - CBR - Nth Pass

The successive passes of a constant bitrate multipass video encoding type are used to optimize the video quality, and are performed after the first pass.

This can be executed any number of times to better optimize the video quality given the desired bitrate. Calling it once gives already very good results. When *Options* button is pressed the following dialog box appears:



- **Stat File:**
  This parameter is already described in previous sections.

- **Preload Time:**
  This value is the time that the decoder should take to preload the stream data before beginning the playback. The higher the value, the better the encoder will be able to optimize the quality of the video. Valid values are between 0 and 3600 seconds, the default value is 1 second.

Multipass Encoding Howto

Here are the basic steps to follow for multipass encoding.

First Encoding:

- Select "Multipass - First Pass" from configuration dialog box.
  At this time you can change the place where to store the "Stat File" by pressing the "More" button.

- Close the dialog box and perform the first encoding.

The Successive Encoding

- Select "Multipass - Next Pass" from configuration dialog box. Set the original source to the second encoding, not the Avi file generated from the first encoding.. Be careful not to use the Avi file that was generated from the first encoding. You can change the place where to read/update the "Stat File" by using the "More" button. Choose the "Stat File" created in the previous encoding pass. If you plan to perform more than two encoding passes, please check the "Update Statistics" check box.

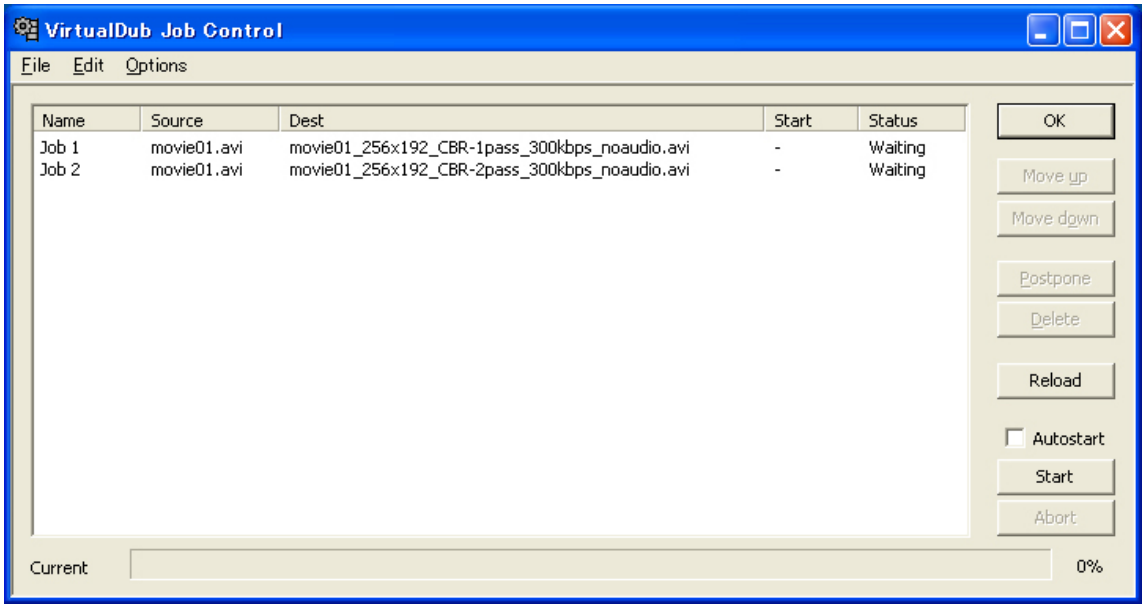
- Close the dialog box and perform the successive encoding passes.

Utilizing Job Control

Please note that under VirtualDub tool you can do all these passes automatically by using its batch job feature.

To create and use batch jobs you have to do the following steps:

- For each encoding pass select [File]->[Save As AVI…] and click on "*Don't run this job now; add it to job control so I can run it in batch mode*" before saving.
- If you are using VirtualDub V1.9.6, for each encoding pass select [File]->[Queue batch operation]->[Save As AVI…].
- Click on [File]->[Job Control…] to manage and launch encoding batches.



Sound Preparation

- Audio should be **normalized** at 100% so it can use the whole dynamic range of the DS.
- Make sure to prepare the AVI sound track using either the **PCM 16 bit mono** format or the **PCM 16 bit stereo** format and a **sampling frequency ranging from 8KHz to 48KHz**. The same constraints apply to external audio files (.wav) used with the AVIToMobiclipDS conversion program.
- When you use the multi-track audio feature:
  - The multi-track audio feature allows you to include multiple audio tracks in a same movie file. For instance, if you include both Japanese and English audio tracks, you can create a movie that can be switched from one language to the other.
  - For detailed instructions about how to make a multi-track audio movie, please refer to the '-audio' parameter description in the 'Conversion Program' chapter.
  - For details about how to play a multi-track audio Mobiclip video file on TWL system, please refer to the 'multitrackAudio' sample code and/or Mobiclip function reference manual.
  - It is not possible to convert an AVI file containing multi-track audio data in a multi-track Mobiclip video file, only the first audio track of the AVI file will be used in this case.

AviToMobiclipDs conversion program

AviToMobiclipDs.exe is a console program used to convert an input avi file containing Mobiclip compressed video frames to a binary file suitable for the Mobiclip libraries. This tool compresses the sound during conversion.

Syntax: AviToMobiclipDs <commands>
Example: AviToMobiclipDs -in inputfile.avi -out outputfile.mods -codebook -quality 256

Commands

The command that can be used for **AviToMobiclipDS.exe** is as follows.

-in <inputfile.avi>
It is a video source(*.avi) that should be converted.

-audio <inputaudiofile.wav>
This option allows you to use external wav files as audio source instead of the embedded sound track of the avi file. In that case, the sound track of the AVI file is ignored.

In order to support multi-track audio this option can be called multiple times, given that all wav files have the same frequency and number of channels.

Example:
AviToMobiclipDS –in in.avi –audio english.wav –audio german.wav –out out.mods.

Cautions:

- If this option is called multiple times, -debugsound option cannot be used.

- It is not possible to convert directly from the AVI file with multi-track audio source.

- Only mono or stereo PCM 16 bits WAV files can be used.

- Basically, there is no limit to the number of audio tracks you can add in a Mobiclip video file. However, the size of the movie will increase according to the number of audio tracks.

- Also 5.2KB of main memory will be used during playback for each audio track added in the Mobiclip file.

-out <outputfile.mods>

The output file name(*.mods) is input. The extension of the final binary file should be “mods”.

-codebook

Always rebuild the audio codebook.

-nocodebook

Never rebuild the audio codebook.

-quality

Audio quality factor, default value is 128, value must be positive. The valid range is 0-1000

-fa

Use FastAudio audio codec instead of the default one. As the bitrate is fixed and no codebook is used in this codec the commands -codebook -nocodebook and –quality are ignored.

-adpcm

Use IMA ADPCM audio codec instead of the default one. As the bitrate is fixed and no codebook is used in this codec, the commands -codebook -nocodebook and –quality are ignored.

-pcm

Use raw PCM audio codec instead of the default one. As the bitrate is fixed and no codebook is used in this codec, the commands -codebook -nocodebook and –quality are ignored.

-debugsound <outputfile.wav>

This option allows you to output the resulting compressed sound in WAV format :useful when you want to have a quick preview of the quality of the compressed sound without the need of a TWL development kit.

NOTE - for the default audio codec

A typically good trade-off between sound quality and data-rate is to use 32.768Hz sound with the default *quality* parameter (128). This should give you a sound data-rate which is about 32kbit/s.

By comparison, an average video data-rate for a full-screen (256x192) high quality (30fps) video can typically be around 400kbit/s (this depends highly on the video).
You should keep in mind the video data-rate when tuning audio quality, as audio generally doesn't use much of the available space. On the other hand, low data-rate video can make audio data-rate more significant.

If you are not satisfied with the sound quality or data-rate, you can adjust both the sampling frequency and the *quality* parameter.

The *quality* parameter ranges from 0 to 1000. Based on the value you enter the resulting data-rates range from 0.6 to 1.25 bit /sample. Low *quality* gives low data-rates; high *quality* gives high data-rates. Good trade-offs between quality and data-rate can be found around the default value (128).

If the sound quality is not good enough at 16kHz with a high *quality* parameter, we recommend increasing sampling frequency at 32kHz (but it will also increase data-rate accordingly).
In the same way, if data-rate is too high, even with a low *quality* parameter, we recommend decreasing sampling frequency.

NOTE - for the Codebook

A codebook (.cbk) file is an "audio analysis" file generated the first time a video is converted.
If the same file is converted another time, the codebook is not re-calculated (which saves time).

The *codebook* option forces the re-calculation of the codebook, even if the video file has not changed.

The *nocodebook* option is useful when you convert a video file which is changing many times always keeping the same audio track (typically when you are trying different video compression parameters on the same file).
If you use the *nocodebook* option, the codebook will not be recalculated every time, it will be done only the first time saving time accordingly.

NOTE - for the FastAudio audio codec

The data-rate is fixed at 1.25 bits/sample. Decompression is about two times slower than the default audio codec but the sound quality can be substantially better.

If you require better sound quality, please use this audio codec and take into consideration the slower decoding speed as well as the potentially larger resulting file size issue.

NOTE - for the ADPCM audio codec

The data-rate is fixed at 4 bits/sample. Decompression is more than two times faster than the default audio codec.

If you require very good sound quality, please use this audio codec and take into consideration the potentially larger resulting file size issue.

NOTE - for the PCM audio codec

This is a lossless codec so the data-rate is fixed at 16 bits/sample and there is no decompression penalty.

If you require the best possible sound quality, please use this audio codec and take into consideration the potentially larger resulting file size issue.

Mobiclip videos

You will find in the *libMobiclip/buid/demos/Mobiclip/data* directory the following Mobiclip resources:

- DS_up.mods
- DS_down.mods
- DS_mono.mods
- DS_stereo.mods
- DS_effectiveUsage.mods
- DS_Multitrack.mods

These resources are used in the sample code described below.

Sample codes

You will find in the *libMobiclip/build/demos/Mobiclip* directory five different code samples:

- *singleScreen* is a basic code sample showing how to use the Mobiclip libraries to play a simple video file with mono sound.
- *dualScreen* is a basic code sample showing how to use the Mobiclip libraries to play two different video files (with mono sound) on the upper and lower screens.
- *singleScreenStereo* is a basic code sample showing how to use the Mobiclip libraries to play a simple video file with stereo sound.
- *effectiveUsage* is a code sample showing how to use the Mobiclip libraries to optimally play a simple video file with mono sound. It uses an alarm callback for the video display management which helps reduce frame skipping and uses the visual quality enhancements features.
- *multitrackAudio* is a code sample that plays a video file containing several audio tracks. Only one audio track is played at once and can be changed interactively.

These five code samples are well documented. You are invited to have a look at them to learn how to use Mobiclip.

Mobiclip libraries and prototype files

Prototype files:

- 'Mobiclip.h' in directory *libMobiclip/include/nitro*.
- 'libMobiclip.h' in directory *libMobiclip/include/nitro/Mobiclip*.

Library files:

- in directory *libMobiclip/lib/ARM9-TS/Debug* for Debug builds*:*
  + libMobiclip.a for Nitro builds
  + libMobiclip.TWL.HYB.a for TWL HYBRIID builds
  + libMobiclip.TWL.LTD.a for TWL LIMITED builds
- in directory *libMobiclip/lib/ARM9-TS/Release* for Release builds*:*
  + libMobiclip.a for Nitro builds
  + libMobiclip.TWL.HYB.a for TWL HYBRIID builds
  + libMobiclip.TWL.LTD.a for TWL LIMITED builds
- in directory *libMobiclip/lib/ARM9-TS/Rom* for Rom builds*:*
  + libMobiclip.a for Nitro builds
  + libMobiclip.TWL.HYB.a for TWL HYBRIID builds
  + libMobiclip.TWL.LTD.a for TWL LIMITED builds

Using the library:

Include the file 'Mobiclip.h' in your source code and link with one the libMobiclip libraries described earlier.
For further details, see the html documentation of Mobiclip and have a look at the sample code.

Annex: Video resources compression settings

Preamble:

The following explaination regards the sample movies distributed within the libMobiclip/build/demos/Mobiclip/data directory. Video sources are PAL videos (720x576 at 25fps). So resulting samples are either 25fps or 12.5fps, but note that you can go up to 30fps in single screen display, and 15fps for dualscreen.

Refer to the chapter “Mobiclip SDK for TWL features” in this document for more details.

DS_up.mods

Resolution: 256x192
Video bitrate: 250Kbps
Framerate: 12.5 fps
Audio: 16000Hz mono, compressed with FastAudio codec (-fa option in AviToMobiclipDs program)
Video encoding process:

- Pass 1:
  - Encoding type: Multipass - VBR - First pass
  - Bitrate: 250
  - All other options to their default values
- Pass 2:
  - Encoding type: Multipass - VBR - Nth pass
  - Bitrate: 250
- Pass 3: same as pass 2

DS_down.mods

Same settings as DS_Up.mods, but without sound.

DS_stereo.mods

Resolution: 256x192
Video bitrate: 350Kbps
Framerate: 25 fps
Audio: 16000Hz stereo, compressed with FastAudio codec (-fa option in AviToMobiclipDs program)
Video encoding process:

- Pass 1:
  - Encoding type: Multipass - VBR - First pass
  - Bitrate: 350
  - All other options to their default values
- Pass 2:
  - Encoding type: Multipass - VBR - Nth pass
  - Bitrate: 350
- Pass 3: same as pass 2

DS_mono.mods

Resolution: 256x192
Video bitrate: 350Kbps
Framerate: 25 fps
Audio: 16000Hz mono, compressed with FastAudio codec (-fa option in AviToMobiclipDs program)
Video encoding process:

- Pass 1:
  - Encoding type: Multipass - VBR - First pass
  - Bitrate: 350
  - All other options to their default values
- Pass 2:
  - Encoding type: Multipass -VBR - Nth pass
  - Bitrate: 350
- Pass 3: same as pass 2

DS_effectiveUsage.mods

Resolution: 256x192
Video bitrate: 185Kbps
Framerate: 25 fps
Audio: 16000Hz mono, compressed with FastAudio codec (-fa option in AviToMobiclipDs program)
Video encoding process:

- Pass 1:
  - Encoding type: Multipass - VBR - First pass
  - Bitrate: 185
  - All other options to their default values
- Pass 2:
  - Encoding type: Multipass -VBR - Nth pass
  - Bitrate: 185
- Pass 3: same as pass 2

DS_Multitrack.mods

Resolution: 256x192
Framerate: 25 fps
Audio: 4 audio tracks 16000Hz, compressed with ADPCM codec (-adpcm option in AviToMobiclipDs program)
Video encoding process:

- Single pass - Constant Quality.
- Quality parameter to 60.0
- Keyframe Maximum interval to 1 second.